

Clase de Consulta - 01/12

Especificación Funcional

$f : [Int] \rightarrow Bool$

$f.xs = \text{"la suma de los números pares de } xs \text{ es } \quad \text{mayor que} \quad \text{la suma de los números impares de } xs \text{"}$

Ejemplo: $f.[2,1,56,67,8,3,4,67] = \begin{matrix} (2 + 56 + 8 + 4) = (1 + 67 + 3 + 67) \\ 70 \qquad \qquad \qquad = \qquad \qquad \qquad 138 \\ \text{False} \end{matrix}$

Herramientas para pensar / chequear una especificación:

1. Dar el tipo. ¿qué toma? ¿qué devuelve?
2. Hacer ejemplos.
3. Analizar la oración que hay que especificar.
- 4. Una vez escrita la especificación, ver si tipa.**

$f.xs = \langle \sum i : 0 \leq i < \#xs \wedge \text{par}.(xs!i) : xs!i \rangle > \langle \sum i : 0 \leq i < \#xs \wedge \text{impar}.(xs!i) : xs!i \rangle$

a donde

$\text{par} : Int \rightarrow Bool$

$\text{par}.i = i \bmod 2 = 0$

$\text{impar} : Int \rightarrow Bool$

$\text{impar}.i = i \bmod 2 = 1$

Pregunta de examen sobre derivación funcional con generalización

$h.xs = \langle \exists as, bs : xs = as ++ bs : sum.as = \#as + 1 \rangle$

Salteamos cosas y vamos derecho al paso inductivo:

$h.(x \blacktriangleright xs)$

= { especificación }

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs : sum.as = \#as + 1 \rangle$

= { 3ro excluido }

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs \wedge (as = [] \vee as \neq []) : sum.as = \#as + 1 \rangle$

= { distrib. y part. rango }

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : sum.as = \#as + 1 \rangle \vee$

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs \wedge as \neq [] : sum.as = \#as + 1 \rangle$

= { cambio var. $as \rightarrow a \blacktriangleright as$ }

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : sum.as = \#as + 1 \rangle \vee$

$\langle \exists a, as, bs : x \blacktriangleright xs = (a \blacktriangleright as) ++ bs \wedge a \blacktriangleright as \neq [] : sum.(a \blacktriangleright as) = \#(a \blacktriangleright as) + 1 \rangle$

= { varias propiedades de listas }

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : sum.as = \#as + 1 \rangle \vee$

$\langle \exists a, as, bs : x = a \wedge xs = as ++ bs : sum.(a \blacktriangleright as) = \#(a \blacktriangleright as) + 1 \rangle$

= { eliminación de variable a }

$\langle \exists as, bs : x \blacktriangleright xs = as ++ bs \wedge as = [] : sum.as = \#as + 1 \rangle \vee$

$\langle \exists as, bs : xs = as ++ bs : \underline{sum.(x \blacktriangleright as) = \#(x \blacktriangleright as) + 1} \rangle$

$= \{ \text{def sum y def \#} \}$
 $\langle \exists \text{ as, bs : } x \triangleright xs = \text{as ++ bs} \wedge \text{as} = [] : \text{sum.as} = \# \text{as} + 1 \rangle \vee$
 $\langle \exists \text{ as, bs : } xs = \text{as ++ bs} : x + \text{sum.as} = (\# \text{as} + 1) + 1 \rangle$
 $= \{ \text{arit.} \}$
 $\langle \exists \text{ as, bs : } x \triangleright xs = \text{as ++ bs} \wedge \text{as} = [] : \text{sum.as} = \# \text{as} + 1 \rangle \vee$
 $\langle \exists \text{ as, bs : } xs = \text{as ++ bs} : x + \text{sum.as} = \# \text{as} + 1 + 1 \rangle$

Acá me trabo. No hay forma de llegar a la H.I. Debo generalizar.

Opción 1: Metemos dos variables

$\text{genh.xs.n.m} = \langle \exists \text{ as, bs : } xs = \text{as ++ bs} : n + \text{sum.as} = \# \text{as} + 1 + m \rangle$
 genh generaliza a h ya que: $\text{genh.xs.0.0} = h.xs$

Opción 2: Si paso restando el 1 para la izquierda y lo junto con el x me queda:

$\langle \exists \text{ as, bs : } xs = \text{as ++ bs} : (\underline{x - 1}) + \text{sum.as} = \# \text{as} + 1 \rangle$

Así que puedo generalizar así:

$\text{genh.xs.n} = \langle \exists \text{ as, bs : } xs = \text{as ++ bs} : n + \text{sum.as} = \# \text{as} + 1 \rangle$
 genh generaliza a h ya que: $\text{genh.xs.0} = h.xs$

Derivación sobre segmentos de listas

Por ejemplo **segmento de suma máxima**:

$f.xs = \langle \text{Max as, bs, cs : } xs = \text{as ++ bs ++ cs} : \text{sum.bs} \rangle$

Salteamos un poco y vamos al paso inductivo:

$f.(x \blacktriangleright xs)$

$= \{ \text{esp.} \}$

$\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs ++ cs : \text{sum.bs} \rangle$

$= \{ \text{3ro excluido} \}$

$\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs ++ cs \wedge (as = [] \vee as \neq []) : \text{sum.bs} \rangle$

$= \{ \text{distrib. y part. rango} \}$

$\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs ++ cs \wedge as = [] : \text{sum.bs} \rangle$ max

$\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs ++ cs \wedge as \neq [] : \text{sum.bs} \rangle$

$= \{ \text{elim. var } as \}$

$\langle \text{Max } bs, cs : x \blacktriangleright xs = [] ++ bs ++ cs : \text{sum.bs} \rangle$ max

$\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs ++ cs \wedge as \neq [] : \text{sum.bs} \rangle$

$= \{ \text{prop. listas} \}$

$\langle \text{Max } bs, cs : x \blacktriangleright xs = bs ++ cs : \text{sum.bs} \rangle$ max

$\langle \text{Max } as, bs, cs : x \blacktriangleright xs = as ++ bs ++ cs \wedge as \neq [] : \text{sum.bs} \rangle$

$= \{ \text{el 2do cuantificador haciendo varios pasos llegamos a la H.I.} \}$

$\langle \text{Max } bs, cs : x \blacktriangleright xs = bs ++ cs : \text{sum.bs} \rangle$ max $f.xs$

$= \{ \text{este me queda molestando, es el mismo problema que el original pero con segmentos iniciales en lugar de segmentos cualquiera.} \}$

MODULARIZO CON **modf.xs** = $\langle \text{Max } bs, cs : xs = bs ++ cs : \text{sum.bs} \rangle$ ("segmento inicial de suma máxima") }

$\text{modf.}(x \blacktriangleright xs)$ max $f.xs$

FALTA: DERIVAR TODO.

Pregunta

Bruno Volpini16:10

yo queria preguntar algo teorico, cuando usamos los metodos para encontrar la invariante, el metodo de tomar terminos de una conjuncion y reemplazar constante por variable. Usamos ambas al mismo tiempo para la invariante? o como seria un ejemplo en que se usa solo tomar terminos de una conjuncion?

Respuesta: Nunca usamos ambos métodos a la vez.

Práctico 7: Ejercicio 1 es con **tomar terminos de una conjuncion**.

El resto son todos reemplazo de constante por variable.

Imperativo

Considerá el siguiente programa anotado, donde $\text{sum.A.p.q} = \langle \sum i : p \leq i < q : A.i \rangle$ satisface la propiedad $\text{sum.A.(p-1).q} = \text{sum.A.p.q} + A.(p-1)$ si $0 < p \leq q$.

Const N: Int, A: Array[0,N) of Int;

Var r,u,n : Int;

{ P: N>0 }

r, u, n = 0, 0, N;

{ INV : r = $\langle N p : n \leq p < N : \text{sum.A.p.N} = 2 \rangle \wedge 0 \leq n \leq N \wedge u = \text{sum.A.n.N}$ }

do n≠0 →

 u = u+A.(n-1);

 { R1 }

 if u=2 →

```

        r = r+1
    [] u≠2 →
        skip
    fi ;
    { R2 }
    n = n-1
od
{ Q: r = ⟨ N p : 0 ≤ p < N : sum.A.p.N = 2 ⟩ }

```

Observación: podemos ver que el invariante se obtuvo aplicando la técnica de reemplazo de ctte por var (reemplazando el 0 por n) y después un fortalecimiento.

Demostración del programa:

1. $\{ INV \wedge n \neq 0 \}$ "cuerpo del ciclo" $\{ INV \}$
2. $INV \wedge \neg (n \neq 0) \Rightarrow Q$ // GARANTIZADO POR TECNICA APLICADA
3. $\{ P \} r, u, n = 0, 0, N \{ INV \}$

Veamos el punto 1:

```

{ INV ∧ n≠0 }
u = u + A.(n-1);
{ R1 }
if u=2 →
    r = r+1
[] u≠2 →
    skip
fi ;
{ R2 }
n = n-1

```

{ INV }

Tengo que dar R1 y R2 tales que valgan:

1.1. { INV $\wedge$ $n \neq 0$ } $u = u + A.(n-1)$ { R1 }

1.2. { R1 } if $u=2 \rightarrow r = r+1$ [] $u \neq 2 \rightarrow \text{skip}$ fi { R2 }

1.3. { R2 } $n = n-1$ { INV }

¿Qué son R1 y R2?

- Sabemos que por definición de wp, si digo que $R2 = \text{wp}.(n:=n-1).\text{INV}$ entonces **vale 1.3.**

Luego calculemos R2:

R2
= { def. R2 }
 $\text{wp}.(n:=n-1).\text{INV}$
= { def. wp para := }
 $r = \langle N \mid p : n-1 \leq p < N : \text{sum}.A.p.N = 2 \rangle \wedge 0 \leq n-1 \leq N \wedge u = \text{sum}.A.(n-1).N$

Falta ver 1.2., que ahora nos queda:

1.2. { R1 } if $u=2 \rightarrow r = r+1$ [] $u \neq 2 \rightarrow \text{skip}$ fi { $r = \langle N \mid p : n-1 \leq p < N : \text{sum}.A.p.N = 2 \rangle \wedge 0 \leq n-1 \leq N \wedge u = \text{sum}.A.(n-1).N$ }

Al igual que hicimos con R2, podemos obtener R1 como la wp.(if fi).R2. Haciendo esto, por definición de wp **va a valer 1.2.**

Luego calculemos R1:

R1
= { def. R1 }
 $\text{wp}.(\text{if fi}).R2$
= { def. wp para el if }
 $(u = 2 \vee u \neq 2) \wedge (u = 2 \Rightarrow \underline{\text{wp}.(r = r+1).R2}) \wedge (u \neq 2 \Rightarrow \underline{\text{wp}.\text{skip}.R2})$
= { def. wp para := y skip }

$$\begin{aligned}
& \underline{(u = 2 \vee u \neq 2)} \wedge (u = 2 \Rightarrow R2(r \leftarrow r+1)) \wedge (u \neq 2 \Rightarrow R2) \\
& = \{ \text{lógica} \} \\
& (u = 2 \Rightarrow R2(r \leftarrow r+1)) \wedge (u \neq 2 \Rightarrow R2) \\
& = \{ \text{sustitución en R2} \} \\
& (u = 2 \Rightarrow r+1 = \langle N p : n-1 \leq p < N : \text{sum.A.p.N} = 2 \rangle \wedge \underline{0 \leq n-1 \leq N \wedge u = \text{sum.A.(n-1).N}}) \\
& \wedge (u \neq 2 \Rightarrow r = \langle N p : n-1 \leq p < N : \text{sum.A.p.N} = 2 \rangle \wedge \underline{0 \leq n-1 \leq N \wedge u = \text{sum.A.(n-1).N}}) \\
& = \{ \text{lógica: } (A \Rightarrow B \wedge C) \wedge (\neg A \Rightarrow D \wedge C) \text{ es lo mismo que } (A \Rightarrow B) \wedge (\neg A \Rightarrow D) \wedge C \} \\
& (u = 2 \Rightarrow r+1 = \langle N p : n-1 \leq p < N : \text{sum.A.p.N} = 2 \rangle) \\
& \wedge (u \neq 2 \Rightarrow r = \langle N p : n-1 \leq p < N : \text{sum.A.p.N} = 2 \rangle) \\
& \wedge 0 \leq n-1 \leq N \wedge u = \text{sum.A.(n-1).N}
\end{aligned}$$

Listo R1 y listo 1.2 y 1.3. Sólo falta verificar:

$$\begin{aligned}
1.1. \{ \text{INV} \wedge n \neq 0 \} \quad u = u + A.(n-1) \quad \{ R1: & (u = 2 \Rightarrow r+1 = \langle N p : n-1 \leq p < N : \text{sum.A.p.N} = 2 \rangle) \\
& \wedge (u \neq 2 \Rightarrow r = \langle N p : n-1 \leq p < N : \text{sum.A.p.N} = 2 \rangle) \\
& \wedge 0 \leq n-1 \leq N \wedge u = \text{sum.A.(n-1).N} \}
\end{aligned}$$

Esta es una terna de hoare que conocemos completa y podemos demostrar. Suponemos las hipótesis $\text{INV} \wedge n \neq 0$ y partiendo de $\text{wp.}(u = u + A.(n-1)).R1$ llegamos a True: EJERCICIO PARA UDS?

Derivación funcional con dos variables cuantificadas

$$f.xs = \langle \sum i, j : 0 \leq i < j < \#xs : xs ! i * xs ! j \rangle$$

Paso inductivo:

$$\begin{aligned} & f.(x \blacktriangleright xs) \\ &= \{ \text{esp.} \} \\ & \langle \sum i, j : 0 \leq i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle \\ &= \{ \text{lógica} \} \\ & \langle \sum i, j : (i = 0 \vee 1 \leq i) \wedge i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle \\ &= \{ \text{distrib. y part. rango} \} \\ & \langle \sum i, j : i = 0 \wedge i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle + \\ & \quad \langle \sum i, j : 1 \leq i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle \\ &= \{ \text{def } \# \} \\ & \langle \sum i, j : i = 0 \wedge i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle + \\ & \quad \langle \sum i, j : 1 \leq i < j < \#xs + 1 : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle \\ &= \{ \text{cambio } i \rightarrow i+1, j \rightarrow j+1 \} \\ & \langle \sum i, j : i = 0 \wedge i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle + \\ & \quad \langle \sum i, j : 1 \leq i+1 < j+1 < \#xs + 1 : (x \blacktriangleright xs)!(i+1) * (x \blacktriangleright xs)!(j+1) \rangle \\ &= \{ \text{arit en el rango, def ! en el termino} \} \\ & \langle \sum i, j : i = 0 \wedge i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle + \\ & \quad \langle \sum i, j : 0 \leq i < j < \#xs : xs!i * xs!j \rangle \\ &= \{ \text{H.I.} \} \\ & \langle \sum i, j : i = 0 \wedge i < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!i * (x \blacktriangleright xs)!j \rangle + f.xs \\ &= \{ \text{elim. variable} \} \\ & \langle \sum j : 0 < j < \#(x \blacktriangleright xs) : (x \blacktriangleright xs)!0 * (x \blacktriangleright xs)!j \rangle + f.xs \\ &= \{ \text{y así} \} \end{aligned}$$

Problemas de Borde

Idea general:

Estoy derivando un ciclo.

Fortalecimiento: Al derivar el cuerpo del ciclo, me sobra una parte que para la que no puedo aplicar hipótesis y además no es programable ni simplificable a algo programable (por ej. un cuantificador).

Problema de borde: Pero si fortalezco, luego al intentar derivar de nuevo, de nuevo no puedo aplicar hip. (por ejemplo, si hicimos cambio de N por n , me sobra algo que se refiere a $(n+1)$).

Solución: El fortalecimiento estaba mal hecho, hay que trabajar más la expresión antes de fortalecer para sacar esa parte ("el borde").

Ejercicio con Borde (Recuperatorio 2do parcial - 27 de noviembre de 2018)

Considere el problema de calcular si un arreglo de enteros tiene dos elementos cuyo producto da 1:

Postcondición: $Q: r = \langle \exists i, j : 0 \leq i < j < N : A.i * A.j = 1 \rangle$

Invariante: Por cambio de constante por variable:

$INV: r = \langle \exists i, j : 0 \leq i < j < n : A.i * A.j = 1 \rangle \wedge 0 \leq n \leq N$

Derivación del cuerpo del ciclo: Suponemos $INV \wedge n \neq N$. Vemos la wp:

$wp.(r, n := E, n+1).INV$

= { def. wp }

$$E = \langle \exists i, j: 0 \leq i < j < n+1 : A.i * A.j = 1 \rangle \wedge \underline{0 \leq n+1 \leq N}$$

= { se va }

$$E = \langle \exists i, j: \underline{0 \leq i < j < n+1} : A.i * A.j = 1 \rangle$$

= { logica y particion de rango }

$$E = \langle \underline{\exists i, j: 0 \leq i < j < n : A.i * A.j = 1} \rangle \vee \langle \exists i, j: 0 \leq i < j \wedge j = n : A.i * A.j = 1 \rangle$$

= { hip. }

$$E = r \vee \langle \exists i, j: 0 \leq i < j \wedge j = n : A.i * A.j = 1 \rangle$$

= { elim. variable }

$$E = r \vee \langle \underline{\exists i: 0 \leq i < n : A.i * \mathbf{A.n} = 1} \rangle \quad // \text{ ACÁ PODRÍA FORTALECER PERO SI LO HAGO NO SALE (PROBLEMA DE BORDE).}$$

// CÓMO ME DOY CUENTA? PORQUE A.n está contado en ese existencial.

// DEBO TRABAJAR MÁS ESTO PARA SACAR EL A.n ANTES DE FORTALECER.

= { Ayuda: $\underline{A.i * A.n = 1}$ sólo puede suceder si ambos valen 1 o ambos valen -1.

¿Qué pasa si $A.n \neq 1$ y también $\neq -1$? Es rango vacío. }

- Caso **$\underline{A.n \neq 1 \wedge A.n \neq -1}$** :

= { Por la ayuda }

$$E = r \vee \langle \exists i: 0 \leq i < n : \text{False} \rangle$$

= { Termino ctte. }

$$E = r$$

= { Elijo **$\underline{E = r}$** }

True

- Caso **$\underline{A.n = 1}$** :

= { Por la ayuda, algún A.i debe ser = 1 }

$$E = r \vee \langle \underline{\exists i: 0 \leq i < n : A.i = 1} \rangle$$

Acá me trabo y debo fortalecer con $s = \langle \exists i: 0 \leq i < n : A.i = 1 \rangle$ (y sin problema de borde) .

- Caso **$\underline{A.n = -1}$** :

= { reemplazo A.n por lo que vale }

$$E = r \vee \langle \exists i: 0 \leq i < n: A.i * (-1) = 1 \rangle$$

= { arit. }

$$E = r \vee \langle \exists i: 0 \leq i < n: A.i = -1 \rangle$$

Acá me trabo y debo fortalecer con $t = \langle \exists i: 0 \leq i < n: A.i = -1 \rangle$ (y sin problema de borde) .

Listo. Acá ya si hago dos los fortalecimientos, puedo derivar de nuevo el cuerpo del ciclo y voy a poder aplicar hipótesis en todos los casos. Nuevo invariante:

$$r = \langle \exists i, j: 0 \leq i < j < n: A.i * A.j = 1 \rangle \wedge 0 \leq n \leq N \wedge s = \langle \exists i: 0 \leq i < n: A.i = 1 \rangle \wedge t = \langle \exists i: 0 \leq i < n: A.i = -1 \rangle$$

Derivación del nuevo cuerpo del ciclo: Arrancamos con wp.(r, s, t, n := E, F, G, n+1). En la derivación, vamos a repetir los mismos pasos incluyendo el análisis por casos. (Ejercicio para uds!).

Resultado:

r, s, t, n := False, False, False, 0 ; // inicialización: rangos vacíos de existenciales

do n ≠ N →

if A.n ≠ 1 ∧ A.n ≠ -1 →

r, s, t, n := False, False, False, n+1

□ A.n = 1 →

r, s, t, n := r ∨ s, True, t, n+1

□ A.n = -1 →

r, s, t, n := r ∨ t, s, True, n+1

fi

od